

Descripción del sistema

Tlapiani (del náhuatl el que custodia o guarda algo) es un ecosistema digital descentralizado diseñado para mitigar la brecha de desigualdad en la distribución de recursos básicos y ayuda humanitaria en zonas de alta marginación rural en México. El sistema transforma el esquema tradicional de donaciones reactivas en un modelo logístico auditable y gobernado por datos, atacando directamente las problemáticas de desvío de insumos e ineficiencia en la asignación reportadas por la Auditoría Superior de la Federación (ASF).

El sistema integra tres capas tecnológicas fundamentales:

- Inteligencia Analítica (Capa de Priorización): Un motor centralizado que elimina la discrecionalidad política mediante la asignación algorítmica de suministros.
- Ciberseguridad e Inmutabilidad (Capa de Integridad): Un mecanismo de certificación que genera un "pasaporte digital" para cada lote utilizando funciones de hashing.
- Accesibilidad en la Última Milla (Capa de Movilidad): Una herramienta portátil diseñada para operar de forma híbrida (offline/online), respondiendo a la brecha de conectividad rural en México.

Lista de requisitos

Requerimientos Funcionales (RF)

Área 1: Backend y Base de Datos (Cerebro y Ciberseguridad)

Este módulo centraliza la lógica de negocio, la seguridad criptográfica y el algoritmo de inteligencia artificial.

RF-1.1: Motor de Priorización (IA): El sistema debe calcular de manera automática el Score de Urgencia de cada comunidad rural utilizando una ponderación de variables estáticas (índices de marginación de CONAPO/CONEVAL) y variables dinámicas (alertas de desastre de CENAPRED).

RF-1.2: Generación de Sello Digital (Hash): Al registrar la salida de un lote de bienes, el backend debe generar un identificador único utilizando el algoritmo SHA-256. Este hash se calculará concatenando los datos del lote: ID_Lote + Tipo_Bien + Cantidad + ID_Comunidad_Destino + Timestamp.

RF-1.3: Arquitectura de API REST: El backend debe exponer endpoints seguros protegidos por autenticación basada en tokens (JWT) para que tanto el Dashboard Web como la App Móvil puedan consumir y enviar datos.

RF-1.4: Persistencia y Trazabilidad: La base de datos (PostgreSQL) debe registrar el historial inmutable de cada envío, manteniendo los estados del ciclo de vida: Creado, En Ruta, Entregado Exitosamente o Alerta de Manipulación.

Área 2: Dashboard Web Administrativo (Gestión y Transparencia)

La interfaz de escritorio utilizada por los administradores de los centros de acopio y auditoría pública.

RF-2.1: Panel de Control de Inventario: Permitir el registro de usuarios (administradores, donantes, transportistas) y la captura detallada de los bienes donados para alimentar el motor criptográfico.

RF-2.2: Visualización Geográfica de Prioridad: Integrar un mapa interactivo de México que pinte las zonas rurales o de emergencia con códigos de color basados en el Score de Urgencia calculado por la IA (ej. Rojo para prioridad crítica, Verde para suministro seguro).

RF-2.3: Despacho de Envíos: Permitir la asignación de lotes de ayuda a rutas específicas y mostrar en pantalla el código QR generado a partir del hash del paquete para su impresión o pegado físico.

RF-2.4: Portal de Transparencia Público: Disponer de un buscador abierto donde cualquier ciudadano o donante pueda ingresar el ID de un lote y visualizar su "pasaporte digital" (historial inalterable de trayecto y confirmación de entrega).

Área 3: Aplicación Móvil (Validación en Campo)

La herramienta portátil utilizada por voluntarios o autoridades locales en las comunidades rurales para cerrar la brecha de la "última milla".

RF-3.1: Escáner de Integridad: Utilizar la cámara del dispositivo móvil para escanear el código QR del paquete al momento de la recepción física en la comunidad.

RF-3.2: Validación Local (Modo Criptográfico): La app debe recalcular el hash SHA-256 de los datos del paquete recibido y compararlo con el hash original firmado por el centro de acopio.

RF-3.3: Base de Datos Local (Modo Offline): Permitir el almacenamiento temporal de registros de entrega y firmas digitales en un almacenamiento local persistente (SQLite o Hive) cuando el dispositivo se encuentre en zonas rurales sin cobertura celular.

RF-3.4: Sincronización Diferida: Al detectar una conexión a internet estable, la aplicación debe enviar automáticamente al servidor central la cola de entregas validadas localmente para actualizar las bitácoras globales.

Requerimientos No Funcionales (RNF)

Afectan a la calidad, rendimiento, seguridad y despliegue del ecosistema tecnológico.

Seguridad y Confidencialidad

RNF-1.1: Inmutabilidad de Datos: El diseño lógico de la base de datos debe impedir la edición o eliminación de registros históricos de entrega una vez que el hash ha sido validado.

RNF-1.2: Cifrado en Tránsito: Toda comunicación entre los clientes (Web/Móvil) y la API de FastAPI debe realizarse obligatoriamente mediante protocolos seguros HTTPS y canales de red protegidos.

Rendimiento y Disponibilidad

RNF-2.1: Tolerancia a Fallos de Red: La aplicación móvil debe mantener su funcionalidad operativa de validación de integridad al 100% de forma local, independientemente de la disponibilidad de red celular en comunidades remotas.

RNF-2.2: Concurrencia de la API: El backend en FastAPI debe ser capaz de procesar múltiples solicitudes asíncronas simultáneas de actualización de inventarios y sincronización de hashes sin degradar el tiempo de respuesta.

Portabilidad y Mantenimiento

RNF-3.1: Contenerización: Toda la infraestructura del backend y la base de datos debe estar completamente empaquetada en contenedores Docker, garantizando que el sistema corra idéntico en entornos de desarrollo local y producción.

RNF-3.2: Compatibilidad de Dispositivos: La aplicación móvil desarrollada en Flutter debe estar optimizada para ejecutarse de manera fluida en smartphones de gama media-baja, reduciendo el consumo de batería y almacenamiento.

Delegación de Roles y Tareas

Integrante 1: Líder de Arquitectura, Ciberseguridad y App Móvil

Es el encargado de asegurar la integridad criptográfica y la herramienta de validación en terreno.

Responsabilidades principales:

- Configurar el entorno global de desarrollo y despliegue utilizando Docker.

- Desarrollar la aplicación móvil en Flutter enfocada exclusivamente en el módulo de escaneo (cámara) y decodificación de datos.
- Implementar la lógica de generación y validación local del Hash SHA-256 dentro de la app móvil para permitir auditorías offline en comunidades con baja conectividad.
- Diseñar la simulación de sincronización de datos diferida (cuando la app detecte red, envía los hashes firmados al servidor central).

Integrante 2: Desarrollador Backend y Datos (Backend & Data Engineer)

Es el encargado del cerebro del sistema: la API, la base de datos y el algoritmo de inteligencia artificial.

Responsabilidades principales:

- Construir la API REST utilizando Python (FastAPI) para asegurar un procesamiento rápido y asíncrono.
- Diseñar y administrar la base de datos relacional en PostgreSQL (tablas para usuarios, inventarios, lotes de donación, catálogos de comunidades rurales e histórico de hashes encadenados).
- Desarrollar el algoritmo del Score de Prioridad (IA): un script interconectado que pondere las variables estáticas de marginación (CONAPO/CONEVAL) y las alertas dinámicas de riesgo.

Integrante 3: Desarrollador Frontend, Integración y Documentación (Fullstack/QA)

Es el encargado de la experiencia web de los administradores y de cumplir con los rigurosos entregables del concurso.

Responsabilidades principales:

- Desarrollar el Dashboard Administrativo Web (interfaz responsiva para los centros de acopio y visualización del mapa de prioridad).
- Consumir los endpoints de FastAPI para conectar la interfaz web con la base de datos.
- Diseñar el panel público de transparencia (donde los donantes pueden consultar el estado de su hash).
- Liderar la redacción técnica del Artículo formato IEEE y asegurar que el Resumen Ejecutivo final esté perfectamente alineado con los hallazgos de la investigación.